

K8s Networking & Configuration for AI Workloads

Session 3 · Week 3 · Services, ConfigMaps & Istio

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

Kubernetes Service Types

ClusterIP

`internal only`

Default. Stable virtual IP inside cluster only. Use for pod-to-pod communication.

NodePort

`dev use`

Exposes service on each node's IP at a static port (30000–32767). Dev/debug only.

LoadBalancer

`production`

Provisions cloud LB (ALB/NLB). Use for production inference endpoints.

Headless

`training / stateful`

No ClusterIP. DNS returns pod IPs directly. Essential for StatefulSets & distributed training.

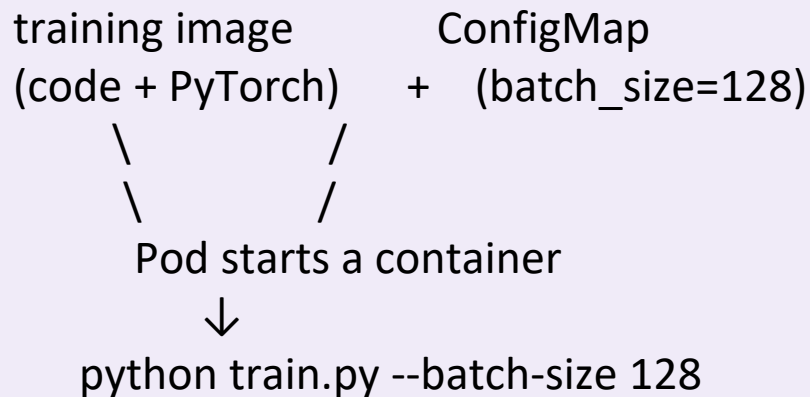
Headless Services are the backbone of distributed training: each training pod gets a stable DNS entry (pod-0.svc, pod-1.svc) needed for NCCL init.

Gateway API v1.2 — The Modern Ingress

Gateway API replaces Ingress as the standard HTTP/gRPC routing layer. Decouples cluster operators (Gateway) from app teams (HTTPRoute).

```
# GatewayClass (platform team)
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: inference-gateway
spec:
  controllerName: istio.io/gateway-controller
---
# HTTPRoute (ML team)
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: llm-inference-route
  namespace: team-alpha
spec:
  parentRefs:
    - name: inference-gateway
  rules:
    - matches:
```

ConfigMaps & Secrets for Training Jobs



same training image + different ConfigMap values = different training run

ConfigMaps & Secrets for Training Jobs

same training image + different ConfigMap values = different training run

ConfigMap — training hynernparams

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: train-config
data:
  batch_size: "128"
  learning_rate: "3e-4"
  max_steps: "50000"
  warmup_steps: "2000"
  model_arch: llama3-8b
```

Secret — W&B / HF tokens

```
# Create secret from literal
kubectl create secret generic \
  ml-tokens \
  --from-literal=WANDB_API_KEY=...
\
  --from-literal=HF_TOKEN=...

# Consume in pod
envFrom:
- configMapRef: { name: train-
  config }
- secretRef:    { name: ml-tokens
  }
```

Production tip

Use External Secrets Operator (ESO) to sync AWS Secrets Manager / Vault → K8s Secret automatically. Never commit secrets to Git.

Istio Service Mesh for Inference Traffic



mTLS

Automatic mutual TLS between all pods. Zero-trust network inside the cluster.



Traffic Split

Canary inference rollouts: route 10% to new model, 90% to stable. VirtualService config.



Observability

Auto-injected metrics (Prometheus), traces (Jaeger), and access logs. No code changes.



Rate Limiting

Envoy-level rate limits protect inference endpoints from request bursts.

```
# 10% canary to new model version
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: llm-inference-vs
spec:
  http:
  - route:
    - destination: { host: llm-svc, subset: stable }
```

Storage for AI Workloads — PVCs & StorageClasses

ReadWriteOnce (RWO)

Single node read-write. Default for NVMe/local SSDs. Fast but not shareable across pods.

ReadWriteMany (RWX)

Multiple nodes read-write simultaneously. Required for shared dataset access during distributed training.

ReadOnlyMany (ROX)

Multiple nodes read-only. Efficient for shared model weights served to many inference pods.

StorageClass examples for AI clusters

- local-nvme: provisioner=local — ultra-fast scratch space for dataset staging
- efs-sc / azurefile-csi: managed NFS — shared dataset volumes across training pods
- weka-sc / vast-csi: high-performance parallel FS — 1M+ IOPS for large-scale training



ConfigMap · train-config

Externalising training hyperparameters from the image

Kind: ConfigMap

API Version: v1

- Stores non-sensitive key-value configuration (batch_size, learning_rate, max_steps, model_arch) outside the container image
- Tuning a hyperparameter is an edit to the ConfigMap, never a rebuild of the training image
- Consumed by training-pod through envFrom → configMapRef, so values land directly in the container's environment



ConfigMap · train-config

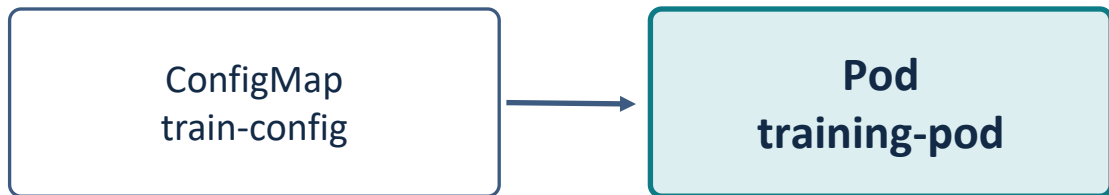
The Manifest

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: train-config
data:
  batch_size: "128"
  learning_rate: "3e-4"
  max_steps: "50000"
  model_arch: llama3-8b
```



ConfigMap · train-config

How It Works



- Data is stored as plain key-value pairs — visible in cleartext, so never use a ConfigMap for secrets
- Any pod in the namespace can reference the same ConfigMap by name
- Changing a value updates the ConfigMap object immediately; the running pod still needs a restart to pick it up

CM

ConfigMap · train-config

Commands & Notes

Apply: `kubectl apply -f 04_configmap.yaml`

Check: `kubectl get configmap train-config -o yaml`

Note: Pair with a Secret (see 05_training_pod / create_secret.sh) for anything sensitive — WANDB/HF tokens never belong in a ConfigMap.



Pod · training-pod

Consuming both a ConfigMap and a Secret via envFrom

Kind: Pod

API Version: v1

- A one-shot training container that pulls its entire environment from two sources: a ConfigMap and a Secret
- Neither hyperparameters nor tokens are baked into the image — tuning a run or rotating a token never requires a rebuild
- restartPolicy: Never — this models a finite training job, not a long-lived service



Pod · training-pod

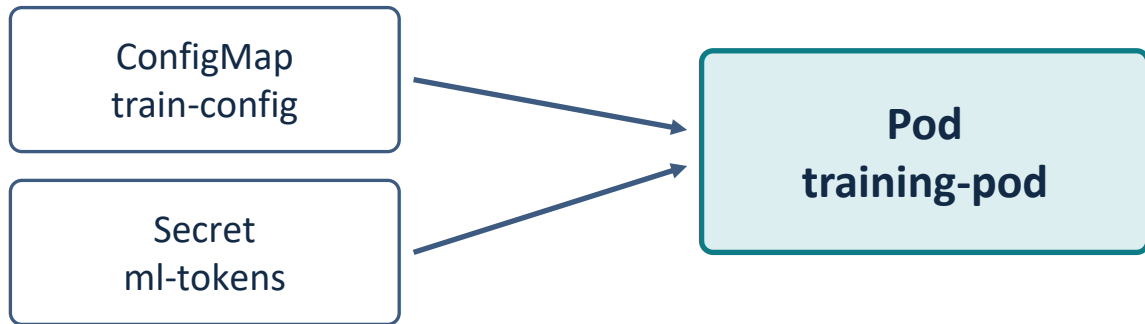
The Manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: training-pod
spec:
  restartPolicy: Never
  containers:
    - name: trainer
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      envFrom:
        - configMapRef:
            name: train-config
        - secretRef:
```



Pod · training-pod

How It Works



- `envFrom` merges every key from both sources straight into the container's environment — no per-key wiring needed
- The Secret (ml-tokens) must exist first — create it with `create_secret.sh` before applying this pod
- Check inside the running container: `kubectl exec -it training-pod -- env | grep -E "batch_size|WANDB|HF_TOKEN"`

Po

Pod · training-pod

Commands & Notes

Apply: `kubectl apply -f 05_training_pod.yaml`

Check: `kubectl exec -it training-pod -- env | grep -E "batch_size|HF_TOKEN"`

Clean: `kubectl delete -f 05_training_pod.yaml`

Note: Run `create_secret.sh` first — the `ml-tokens` Secret this Pod references must already exist, or the Pod stays stuck in `CreateContainerConfigError`.



Headless Service · training-worker

clusterIP: None — stable per-pod DNS instead of one virtual IP

Kind: Service
(headless)

API Version: v1

- clusterIP: None means no virtual IP is allocated for this Service at all
- A DNS lookup on the Service name returns every pod's IP directly, not a single load-balanced address
- Each pod additionally gets its own stable hostname record: <pod-name>.training-worker



Headless Service · training-worker

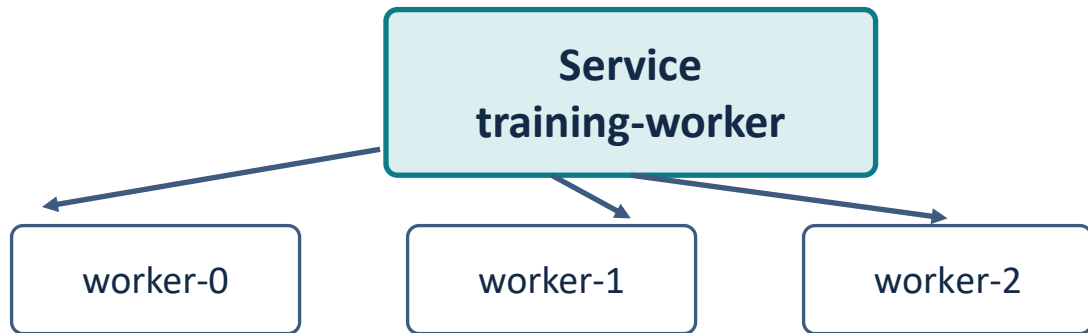
The Manifest

```
apiVersion: v1
kind: Service
metadata:
  name: training-worker
spec:
  clusterIP: None
  selector:
    app: training-worker
  ports:
    - name: nccl
      port: 29500
      targetPort: 29500
```



Headless Service · training-worker

How It Works



- This is the backbone of distributed training: NCCL rendezvous needs every worker to address every other worker by a stable hostname
- A regular ClusterIP Service would resolve all workers to one virtual IP, and NCCL would hang at initialization
- Pairs with the StatefulSet on the next slide, which gives each pod the stable ordinal name this DNS scheme relies on



Sv

Headless Service · training-worker

Commands & Notes

Apply: `kubectl apply -f 06_headless_svc.yaml`



StatefulSet · worker

Ordinal, stable pod identity for distributed training

Kind: StatefulSet

API Version: apps/v1

- 3 replicas get stable, ordinal-indexed names — worker-0, worker-1, worker-2 — instead of a Deployment's random hash suffixes
- Bound to the training-worker headless Service via serviceName, so each pod inherits a predictable DNS record
- Pod identity survives rescheduling: a replacement worker-1 keeps its name, its ordinal, and its DNS entry



StatefulSet · worker

The Manifest

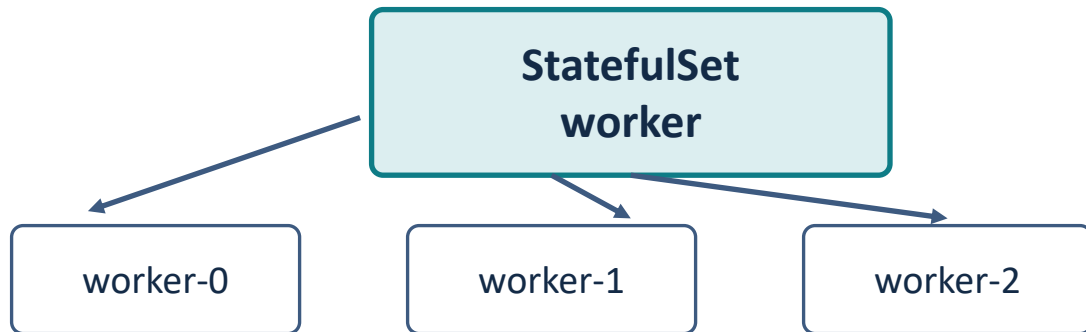
```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: worker
spec:
  serviceName:
training-worker
  replicas: 3
  selector:
    matchLabels:
      app: training-
worker
```

```
    template:
      metadata:
        labels:
          app: training-
worker
      spec:
        containers:
          - name: worker
            image:
busybox:1.36
            command:
["sh", "-c",
  "sleep
3600"]
```



StatefulSet · worker

How It Works



- Each pod is individually addressable at `worker-<N>.training-worker.<namespace>.svc.cluster.local`
- Verify peer discovery: `kubectl exec -it worker-0 -- nslookup worker-1.training-worker`
- Ordinal startup order (0, then 1, then 2) matters for rendezvous protocols that expect a rank-0 coordinator



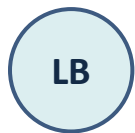
StatefulSet · worker

Commands & Notes

Apply: `kubectl apply -f 07_statefulset.yaml`

Check: `kubectl exec -it worker-0 -- nslookup worker-1.training-worker`

Clean: `kubectl delete -f 07_statefulset.yaml`



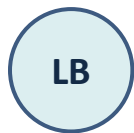
LoadBalancer Service · tiny-inference-lb

Exposing an inference endpoint outside the cluster

Kind: Service
(LoadBalancer)

API Version: v1

- type: LoadBalancer requests an externally-reachable IP from the cluster's cloud or local LB provider
- The production pattern for a real inference endpoint, as opposed to ClusterIP (internal-only) or NodePort
- Routes external traffic on port 8080 to the tiny-inference Deployment's pods on containerPort 8000

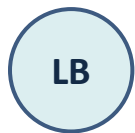


LB

LoadBalancer Service · tiny-inference-lb

The Manifest

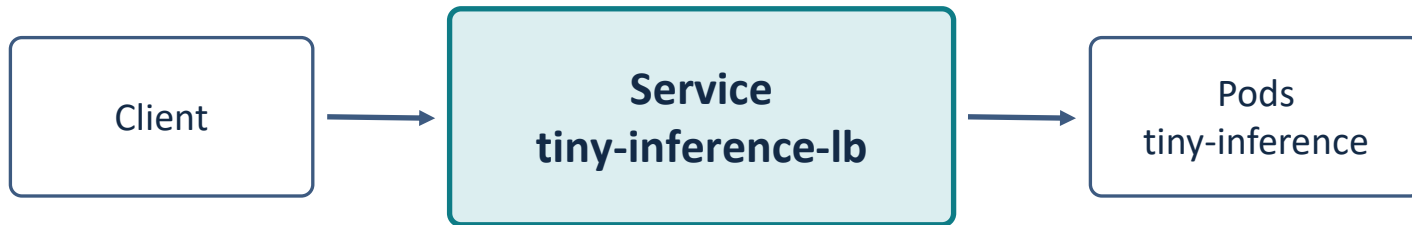
```
apiVersion: v1
kind: Service
metadata:
  name: tiny-inference-lb
spec:
  type: LoadBalancer
  selector:
    app: tiny-inference
  ports:
    - port: 8080
      targetPort: 8000
```



LB

LoadBalancer Service · tiny-inference-lb

How It Works



- Requires the tiny-inference Deployment (../01_inference_service) to already be running — the selector has nothing to route to otherwise
- On Docker Desktop's local Kubernetes, LoadBalancer Services publish on localhost automatically — no cloud LB controller needed
- On a real cluster this provisions an actual external IP via the cloud provider (or MetalLB on bare metal)

LB

LoadBalancer Service · tiny-inference-lb

Commands & Notes

Requires: `kubectl apply -f
../01_inference_service/tiny-inference-
deployment.yaml`

Apply: `kubectl apply -f 08_loadbalancer_svc.yaml`

Check: `kubectl get svc tiny-inference-lb ; curl
http://localhost:8080/health`

Clean: `kubectl delete -f 08_loadbalancer_svc.yaml`



Gateway · inference-gateway

The concrete listener an HTTPRoute attaches to

Kind: Gateway

API Version:
gateway.networking.
k8s.io/v1

Reference: Session
2, slide 6
(supporting)

- References a GatewayClass and, once created, makes Istio provision a real Envoy proxy plus a LoadBalancer Service
- Listens on port 8082 for HTTP — not port 80, which Docker Desktop's own engine already holds on localhost
- `allowedRoutes.namespaces.from`: Same restricts attachment to HTTPRoutes in this Gateway's own namespace (team-alpha)



Gateway · inference-gateway

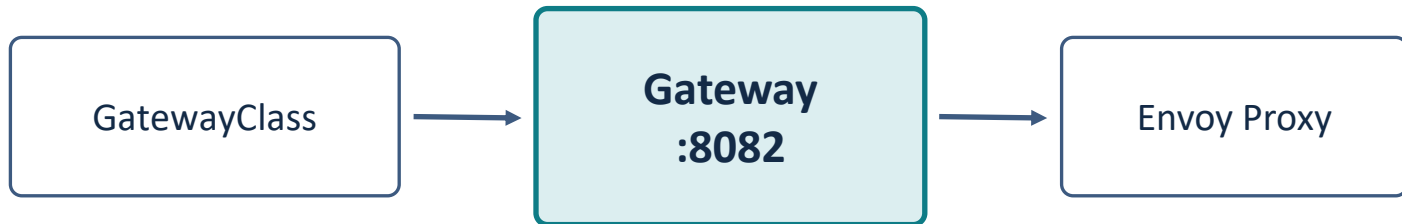
The Manifest

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: inference-gateway
  namespace: team-alpha
spec:
  gatewayClassName: inference-gateway
  listeners:
    - name: http
      port: 8082
      protocol: HTTP
      allowedRoutes:
        namespaces:
          from: Same
```



Gateway · inference-gateway

How It Works



- The HTTPRoute (next deck) attaches to this Gateway via parentRefs to become live traffic-routing config
- Before applying, confirm port 8082 is actually free: `Get-NetTCPConnection - LocalPort 8082 -State Listen`
- 8082 was chosen over 8080/8081 since this course's tiny-inference-lb and other local tools already use those

GW

Gateway · inference-gateway

Commands & Notes

Apply: `kubectl apply -f gateway.yaml`

Check: `kubectl get gateway -n team-alpha inference-gateway`

Note: On a real cluster (cloud LB or MetalLB) every LoadBalancer Service gets its own IP, so the port-8082 workaround is a Docker-Desktop-only quirk — don't present it as universal.



GatewayClass · inference-gateway

Gateway API's replacement for the Ingress class

Kind: GatewayClass

API Version:

gateway.networking.
k8s.io/v1

- Gateway API is the standard HTTP/gRPC routing layer that supersedes Ingress
- One GatewayClass per cluster controller — created once by the platform team; application teams never touch it
- controllerName must match whatever controller the service mesh (here, Istio) actually registers on the cluster



GatewayClass · inference-gateway

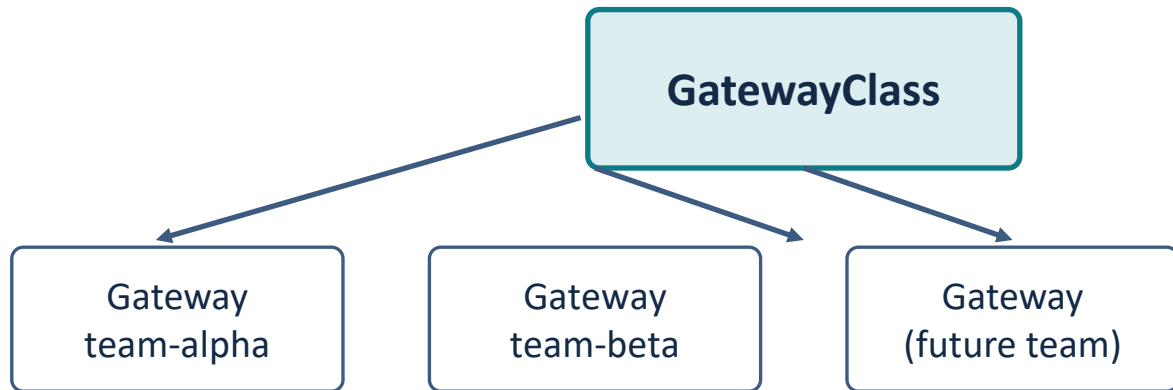
The Manifest

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: inference-gateway
spec:
  controllerName: istio.io/gateway-controller
```



GatewayClass · inference-gateway

How It Works



- Every Gateway in the cluster references a GatewayClass by name (gatewayClassName) to pick its implementation
- Verify the controller Istio actually registered before relying on this: `kubectl get gatewayclasses -o wide`



GatewayClass · inference-gateway

Commands & Notes

Apply: `kubectl apply -f gatewayclass.yaml`

Check: `kubectl get gatewayclass inference-gateway`



HTTPRoute · llm-inference-route

Namespaced, self-service routing — no cluster-admin required

Kind: HTTPRoute

API Version:

gateway.networking.
k8s.io/v1

- Defines a path-based routing rule (/v1/completions) and attaches to a Gateway through parentRefs
- backendRefs sends matching traffic on to the llm-service Service, port 8080
- Created by the ML team inside their own namespace (team-alpha) — unlike classic Ingress, no cluster-admin access is needed



HTTPRoute · llm-inference-route

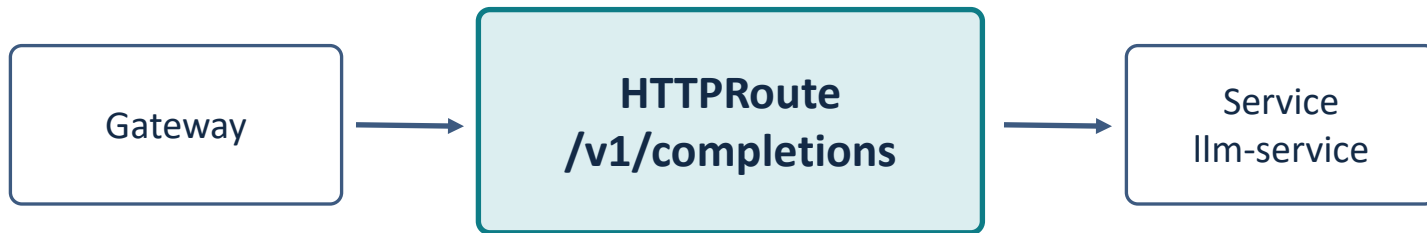
The Manifest

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: llm-inference-route
  namespace: team-alpha
spec:
  parentRefs:
    - name: inference-gateway
  rules:
    - matches:
        - path:
            value: /v1/completions
      backendRefs:
        - name: llm-service
```



HTTPRoute · llm-inference-route

How It Works



- The gatewayClassName and Gateway are platform-owned; only this HTTPRoute lives in the application team's namespace
- Multiple HTTPRoutes from different teams can attach to the same Gateway, each matching its own path
- Traffic-splitting / canary rules (Istio) build on top of this same HTTPRoute resource

HR

HTTPRoute · llm-inference-route

Commands & Notes

Apply: `kubectl apply -f httproute.yaml`

Check: `kubectl get httproute -n team-alpha llm-inference-route`